CANEMAP SYSTEM MANUAL

The CaneMap System Manual presents the technical documentation of the CaneMap platform, including its architectural design, functional modules, technologies used, and deployment structure. Unlike the User/Admin Manual, this document focuses on the internal workings of the system, covering system flow, data structures, and the overall technical environment that supports field registration, user role management, task delegation, and verification workflows.

This manual serves as a reference for developers, IT personnel, and stakeholders who require a deeper understanding of how the CaneMap system operates behind the user interface.

2.0 System Overview

CaneMap is a web-based and mobile-accessible application designed to centralize sugarcane field operations. It coordinates multiple roles, Handler, Worker, Driver, SRA Officer, and Administrator, within a single integrated platform. Core system functions include field registration, geolocation mapping via polygon drawing, work logging with real-time image capture, document management, driver verification, and SRA approval processes.

The system follows a modular-component approach, allowing each role to interact only with system functions relevant to their responsibilities. Data is stored in Firebase Firestore, while Firebase Authentication manages account security and multi-role access.

3.0 System Objectives

The CaneMap system is designed with the following technical objectives:

- To develop a cloud-based system architecture that supports real-time data synchronization.
- To implement secure authentication mechanisms for multi-role access.
- To automate field registration and verification workflows.
- To provide GPS-based field boundary mapping through polygon drawing tools.

- To support structured work logging with timestamped records and media attachments.
- To implement document management and review modules for compliance.
- To create scalable features suitable for future system enhancements.

4.0 System Architecture

The CaneMap system architecture follows a **client–cloud model** using Firebase services for backend operations. Data flows from user interfaces through authenticated requests into Firestore collections.

4.1 High-Level Architecture Description

- **Frontend Layer:**
  Built using HTML, CSS, JavaScript, and Firebase SDKs.
  Accessible via web browsers and Android WebView-based app.

- **Backend Layer:**
  Firebase Authentication
  Firebase Firestore (NoSQL Database)
  Firebase Cloud Functions (optional triggers)
  Firebase Hosting

- **Storage Layer:**
  Firebase Storage for handling uploaded images (work logs, documents, badges).

- **Security Layer:**
  Firebase Authentication
  Firebase Security Rules
  Admin PIN protection for system-level access

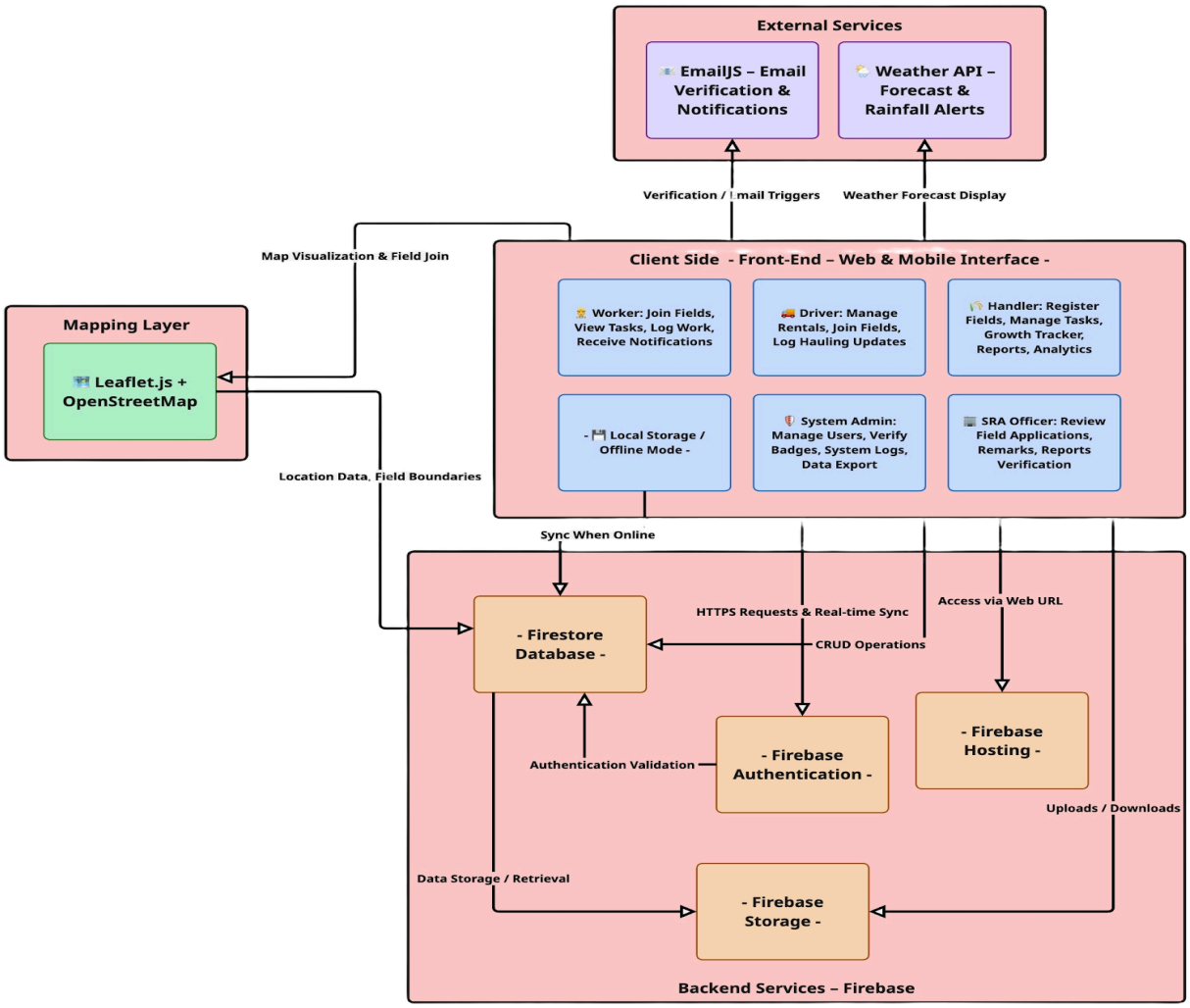

FIGURE 4.1: *System Architecture Overview Diagram*

4.2 Data Flow Between Components

1. User logs in via Firebase Authentication.
2. Auth token validates role and permissions.
3. User performs actions (e.g., submit field, task, work log).
4. Frontend sends data to Firestore collections.
5. Firebase triggers update real-time UI changes.
6. Storage handles image uploads and retrieval.

7. Admin and SRA interfaces read structured data from Firestore.

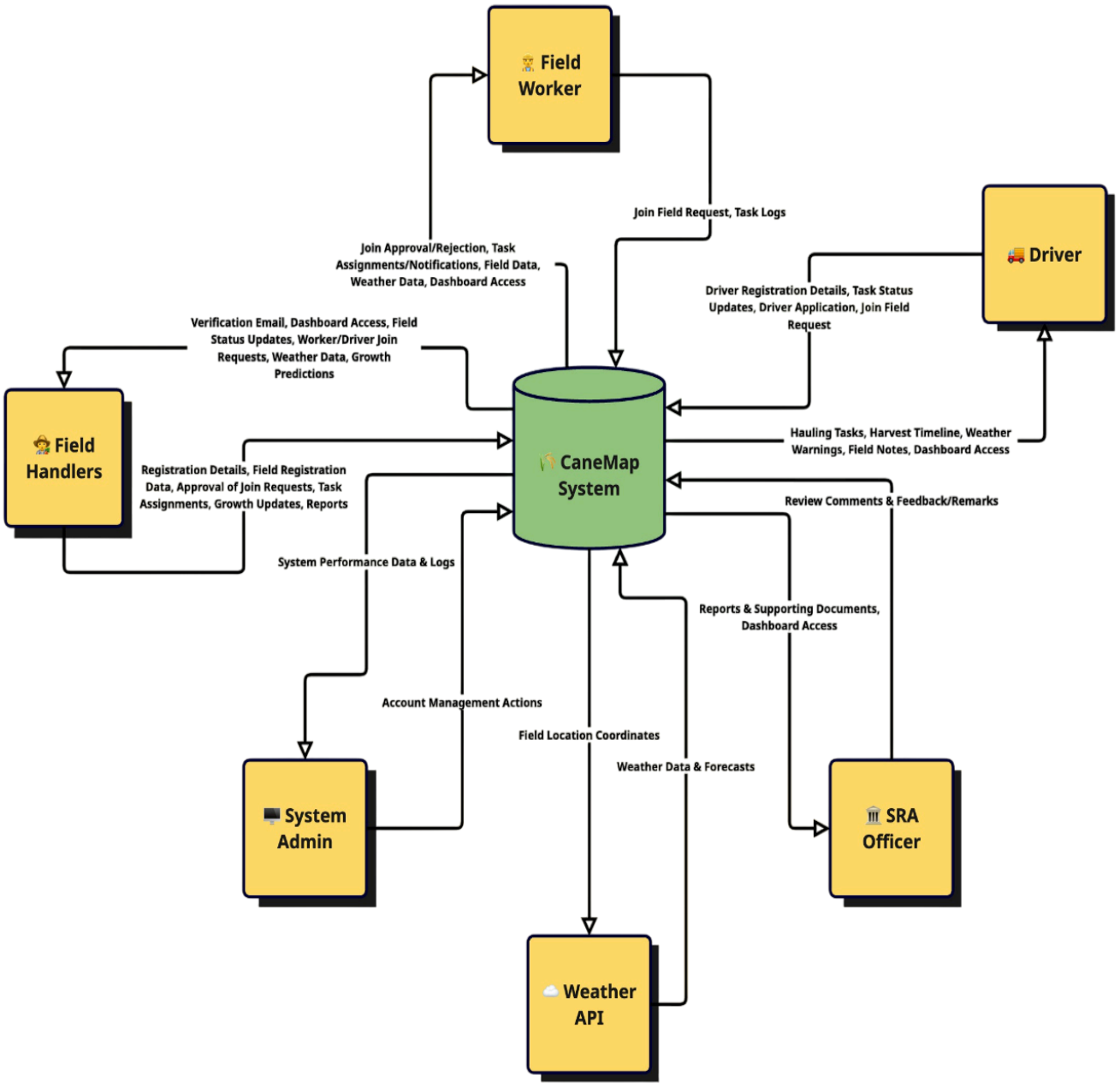

FIGURE 4.2: *Data Flow Diagram – Context-Level*

5.0 Use Case Description

Use cases describe how each role interacts with the system at a functional level. These interactions were extracted from the system flow and interface screenshots.

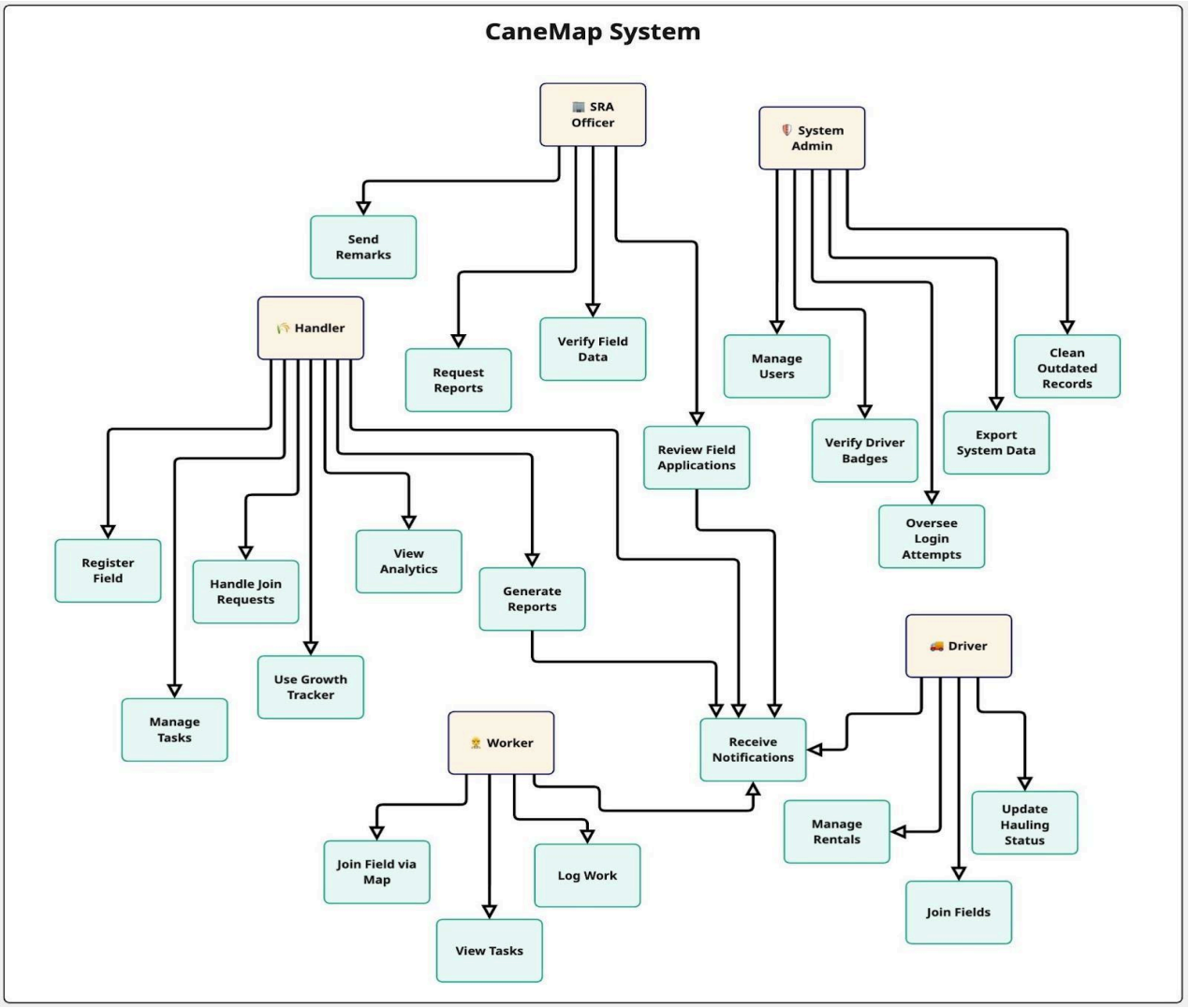

FIGURE 5.0: *Use Case Diagram – CaneMap*

5.1 Major Use Cases

Handlers

- Register Field
- Approve Members
- Create Tasks
- Review Work Logs
- Upload Documents

- Monitor Growth
- Generate Reports

Workers

- Join Field
- View Tasks
- Submit Work Logs

Drivers

- Apply for Badge
- Start/End Transport Tasks
- Submit Transport Logs

SRA Officers

- Review Field Applications
- Approve/Reject Documents

Admin

- Login via Admin Portal
- Change Admin PIN
- Manage User Accounts
- View Activity Logs

6.0 System Flow Descriptions

This section summarizes how the CaneMap system processes major workflows from input to output.

6.1 Field Registration Flow

1. Handler submits field information, polygon, and documents.
2. SRA receives the submission.
3. SRA verifies documents and polygon details.
4. SRA approves or requests revision.
5. Approved fields are marked “Verified.”

6.2 Task Assignment Flow

1. Handler creates a task.
2. Assigned Worker/Driver receives notification.
3. User completes the activity and submits log.
4. Handler reviews and approves logs.
5. Activity appears in field activity records.

6.3 Document Review Flow

1. Handler uploads compliance documents.
2. SRA views and validates submissions.
3. SRA marks as **Approved**, **Rejected**, or **For Revision**.
4. Handler updates and resubmits if required.

6.4 Work Log Flow

1. User opens task.
2. Takes real-time photo.
3. Adds notes and submits log.
4. Log stored with timestamp.
5. Handler verifies log.

7.0 Technology Stack

Frontend:

- HTML5
- CSS3
- JavaScript
- Firebase SDK
- Responsive design for web and Android app

Backend & Infrastructure:

- Firebase Authentication
- Firebase Firestore
- Firebase Hosting
- Firebase Storage

Mapping Tools:

- JavaScript-based polygon mapping (Google Maps / Leaflet JS if used)

Other Tools:

- Git for version control
- Browser-based debugging tools

8.0 Database Design

CaneMap uses a **NoSQL (Firestore)** structure organized into collections.

Core Collections:

- **users** – all users, roles, permissions
- **fields** – field info, polygon data, ownership
- **tasks** – assigned tasks with metadata
- **logs** – work logs with timestamps and images
- **documents** – verification files
- **rentals** – driver rental tracking
- **driver_badges** – badge applications and status
- **notifications** – system alerts per user

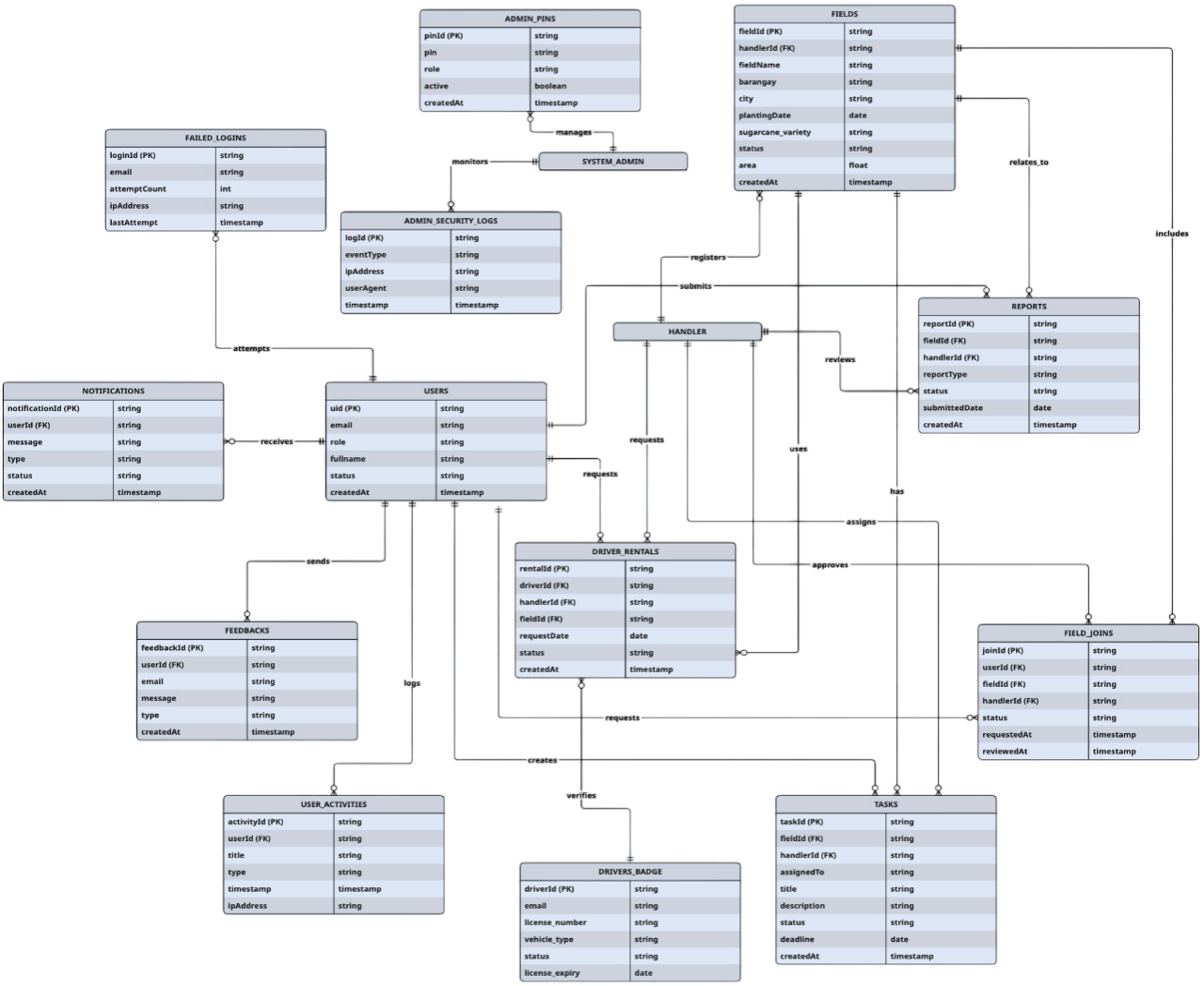

FIGURE 8.0 *Entity Relationship Diagram Model for CaneMap*

9.0 System Modules (Technical Perspective)

Below is each module described from a technical—not user—perspective.

9.1 Authentication Module

- Email–password login
- Role-based access control

- Firebase Authentication tokens
- Admin PIN-based separate entrance

9.2 Field Module

Handles field data creation, polygon storage, updates, and verification logs.

Data stored in:
`fields/{fieldID}` document containing:

- fieldName
- coordinates[]
- documents[]
- handlerID
- verificationStatus

9.3 Task Module

Stores assigned tasks per field/user.

`tasks/{taskID}` contains:

- assignedTo
- fieldID
- description
- schedule
- status

9.4 Work Log Module

logs/{logID} contains:

- taskID
- imageURL
- timestamp
- notes
- approvedStatus

9.5 Document Module

The Document Module handles document uploads, metadata tracking, and verification status management. Files (e.g., land titles, barangay certificates, IDs, and activity photos) are stored in Firebase Cloud Storage while descriptive metadata and version records are stored in the `documents` Firestore collection. Each document record includes fields for `uploaderID`, `fieldID` (when applicable), `fileURL`, `uploadedAt` (timestamp), `version` (integer), `validationStatus` (e.g., `pending`, `approved`, `rejected`), and `reviewerComments`. Versioning is supported by incrementing the `version` field and retaining previous `fileURL` entries so auditors can view historical submissions. Firestore security rules restrict who may upload, update, or approve documents based on user role and ownership.

9.6 Growth Tracker

The Growth Tracker stores crop stage records, planting dates, and calculated harvest estimates for each field. Each field's growth document contains `plantingDate`, `growthStages` (an array of stage records with `stageName`, `recordedAt`, and optional notes), and `harvestEstimate` (a computed date derived from planting date and crop-cycle parameters). Historical stage logs are retained to provide progress timelines, and handlers may add manual notes or corrections. Exportable growth summaries (PDF/CSV) are generated from these records for reporting and auditing purposes.

9.7 Driver Rental & Transport Module

- Rental start/end logs
- Transport task completion
- Driver badge metadata

9.8 Notification Module

Notifications are managed through a **centralized `notifications` collection** in Firestore that stores all system alerts for all users. Each notification document contains `recipientID` (or `recipientRole`/`fieldID` for broadcast messages), `type` (e.g., `task_assigned`, `field_approved`), `message`, `createdAt`, `isRead` (boolean), and an optional `payload` (link to task, field, or document). Client applications listen for changes or query this centralized collection to display relevant alerts; listeners are scoped to the current user or role for efficiency. Firestore security rules enforce that only authorized system processes and permitted user roles may write certain notification types (e.g., only handlers or system processes may send `task_assigned` notifications), while recipients may mark their notifications as read.

9.9 Admin Module

The Admin Module centralizes all system-wide administrative capabilities and provides tools for maintaining operational integrity and security. Access to the Admin Dashboard requires entering the **System Admin PIN**, which by default is set to **123456**. This PIN serves as an additional security layer on top of Firebase Authentication. Administrators may change this PIN at any time through the built-in PIN reset workflow found on the admin sidebar, ensuring that system access remains controlled and secure.

Core functions of the Admin Module include **System PIN management**, where the admin verifies access and updates the PIN as needed; **User management**, which allows administrators to create accounts, update user information, suspend or reactivate accounts, change roles, and perform password resets through entries stored in the `users` collection;

and **Driver badge and document approvals**, where the admin may validate identity and compliance submissions. The module also includes **Activity Log views**, providing exportable audit trails containing login records, role changes, document approvals, and field verification actions. All administrative actions are logged with timestamps and admin identifiers. Firestore Security Rules ensure that only accounts with the admin role, authenticated via the current PIN, can perform or access these privileged operations.

10.0 Security Features

CaneMap integrates security through multiple layers:

Authentication

- Firebase email/password authentication
- PIN-gated admin access

Authorization

- Firestore rules enforce role-based data access
- Users cannot access data belonging to other roles

Data Handling

- HTTPS enforced
- Image uploads secured via signed URLs
- NoSQL validation rules prevent unauthorized writes

Activity Monitoring

- Firestore logs
- Admin review tools

- Timestamped logs for traceability

11.0 Deployment Environment

CaneMap is deployed using:

- **Firebase Hosting** for web
- **Firebase Firestore** for realtime database
- **Firebase Auth** for identity management
- **Android WebView App** (PWA-style build)

Deployment includes:

- Code upload via Firebase CLI
- Firestore rules configuration
- Hosting build & deploy commands
- Linking app domain to hosting service

12.0 Maintenance Guidelines

Regular Maintenance

- Monitor Firestore usage
- Review Firebase Storage for file buildup
- Update Firestore rules as needed
- Review admin logs for suspicious activity

Backup Procedures

- Manual export of Firestore collections
- Backup of Storage files
- Download user reports and logs regularly

System Updates

- Update frontend scripts
- Re-deploy through Firebase CLI
- Review version changes across dependencies

13.0 Limitations of the System

Technical limitations include:

- Requires stable internet connection
- No offline caching for tasks/logs
- Limited to Android for mobile app version
- Dependent on Firebase free-tier constraints

14.0 Future System Enhancements

- Offline-ready features using local storage
- Push notifications via Firebase Cloud Messaging
- AI-based yield prediction tools

- Blockchain document verification